

LoopHire: Loop Engineering-Based Intelligent Tech Stack Recommendation System for Recruitment

Problem Statement

Companies post Job Descriptions (JDs) on platforms like LinkedIn, Naukri, or their career pages.

However:

- HR teams sometimes write incomplete JDs.
- Similar technologies are written differently (e.g., JavaScript vs JS).
- Important skills may be omitted.
- There is no indication of the recommended learning path for candidates.
- AI-generated recommendations can miss relevant technologies or suggest outdated ones.

Current AI systems usually generate recommendations in a single pass without verifying them.

Proposed Solution

Develop a **Loop Engineering-based system** that continuously analyzes a job description, validates the extracted technologies, identifies missing skills, and refines the recommendations until a quality threshold is reached.

The system provides:

- Extracted skills
 - Missing technologies
 - Technology roadmap
 - Learning priority
 - Suggested certifications
 - Confidence score
-

Real-Time Scenario

Imagine Microsoft posts a job:

Software Engineer

Requirements:

- Python , FastAPI, SQL, REST APIs, Docker.

A fresher uploads the JD.

The system returns:

Required Skills

✓ Python

✓ FastAPI

✓ SQL

✓ REST API

✓ Docker

Recommended Skills

✓ Git

✓ Linux

✓ CI/CD

✓ Kubernetes

✓ Azure

Learning Order

Python -> Git -> SQL -> FastAPI -> RestAPI -> Docker -> CI/CD -> Azure -> Kubernetes

Instead of only extracting keywords, the system explains **why** each technology matters.

Why Loop Engineering?

Normally,

JD -> LLM -> Technology List

Done.

If technologies are missing, the process ends there.

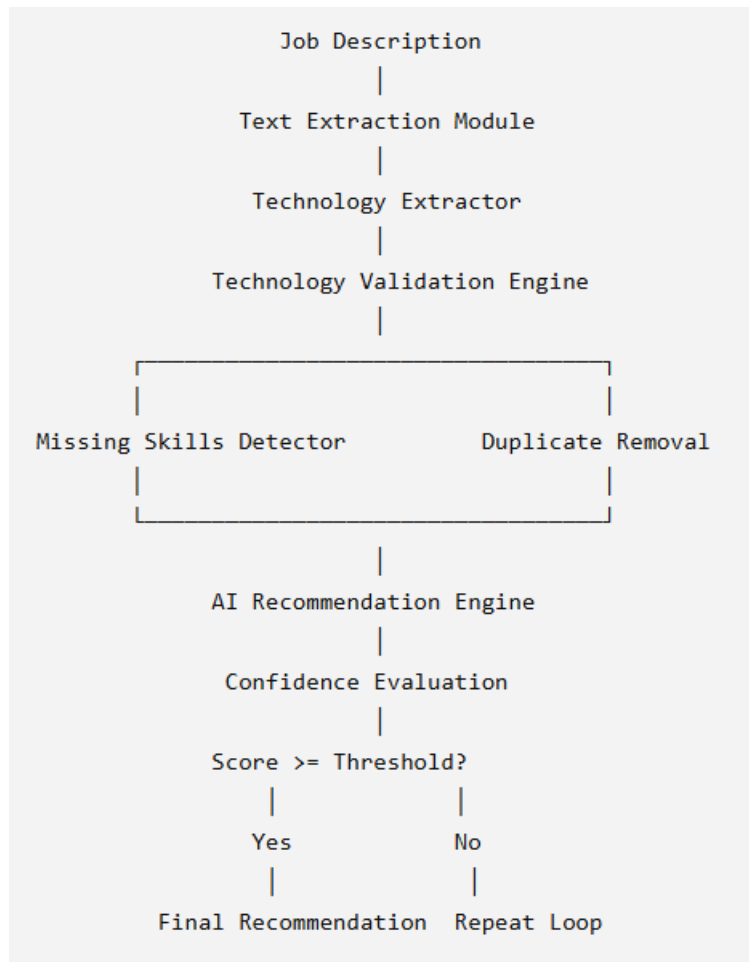
Proposed method:

Job Description -> Extract Skills -> Validate Skills -> Check Missing Technologies ->

Compare with Similar Job Descriptions -> Generate Recommendations -> Evaluate Recommendation Score -> Score > 90 -> No -> Improve Recommendation -> Repeat -> Final Output

The recommendation quality improves with each iteration.

Loop Architecture



Proposed Modules

Module 1 — JD Upload

User uploads:

- PDF
- DOCX
- Text

Example:

Python Developer

Experience: 2 Years

Skills : Python, FastAPI, Docker, SQL, REST APIs

Module 2 — Skill Extraction

Extract:

Programming Languages

Frameworks

Databases

Cloud

Tools

Soft Skills

Output :

Languages, Python, Framework, FastAPI, Database, SQL, DevOps, Docker

Module 3 — Technology Validation

Check:

Is FastAPI related to Python?

YES

Is Docker useful?

YES

Is Git missing?

YES

Module 4 — Missing Skill Detection

System compares against industry knowledge.

Output : Recommended, Git, Linux, Azure, CI/CD, Kubernetes

Module 5 — Learning Path Generator

Instead of random ordering,

AI builds dependencies.

Example

Python -> Git -> SQL -> FastAPI -> REST API -> Docker -> Azure -> Kubernetes

Module 6 — Loop Controller

Evaluate:

Coverage

Completeness

Confidence

Missing technologies

Consistency

If score < 90

Repeat

Algorithm

while score < threshold:

 extract_skills()

 classify_skills()

 validate_skills()

 detect_missing()

 generate_learning_path()

 calculate_confidence()

 improve_recommendations()

Evaluation Metrics

Metric	Description
Technology Extraction Accuracy	Correctly extracted skills
Precision	Correct recommendations
Recall	Missing skills identified
F1 Score	Overall extraction quality
Confidence Score	AI confidence
Number of Iterations	Loop count
Response Time	Execution time

Expected Output

Job Role : Backend Developer

Required Skills: Python, FastAPI, Docker, REST, SQL

Recommended Skills : Git, Linux, Azure, Kubernetes, CI/CD,

Learning Path:

Python -> Git -> SQL -> FastAPI -> Docker -> Azure -> Kubernetes

Overall Score :94%

Research Contribution

Most existing systems:

Input -> AI -> Output

Proposed framework:

Input -> Extract -> Validate -> Improve -> Re-evaluate -> Repeat -> Final Recommendation

The key contribution is the **iterative improvement loop**, which aims to increase the completeness and usefulness of technology recommendations compared with a single-pass approach.

Novel Feature (This is where idea becomes unique)

Add one more module that makes the work stand out.

Company Tech Stack Predictor

Besides recommending technologies, the system predicts:

Company Maturity Level

Example:

Startup

Expected Stack: FastAPI, React, Docker, MongoDB, GitHub Actions

OR

Enterprise

Expected Stack: Java, Spring Boot, Oracle, Kafka, Kubernetes, Azure, Jenkins

Company-wise Learning Path

Example:

TCS

Java, Python, Full-stack, Problem solving

Microsoft

C#, .NET, Azure, SQL Server, Git, Docker

Google

Python, Go, Kubernetes, GCP, TensorFlow

Amazon

Java, AWS, Spring Boot, DynamoDB, Lambda

Zoho

Java, MySQL, JavaScript, Linux

Git

This transforms the PoC from a simple JD parser into an **AI Career Advisor**, making it more compelling for both research and industry demonstrations.

A note on implementation

For the "company-specific stack" feature, avoid hardcoding assumptions about companies. Instead, build a **knowledge base** from:

- public job descriptions,
- official company technology blogs,
- engineering career pages,
- and validated technology mappings.

Then use the loop to:

1. Extract technologies from the uploaded JD.
2. Compare with the knowledge base for similar roles.
3. Identify missing but commonly expected skills.
4. Re-rank recommendations based on confidence.
5. Repeat until the recommendation confidence stabilizes.

This approach is more scientifically defensible and gives a stronger methodology section in this idea because the recommendations are grounded in data rather than fixed rules.